

Groomsta Frontend Structure Document

1. Project Overview

Groomsta is a hybrid grooming platform with three distinct user interfaces:

- **Customer Web App** - Service discovery, booking, and tracking
- **Partner Web App** - Job management and earnings
- **Admin Dashboard** - Platform configuration and monitoring

Technology Stack:

- React / Next.js (SSR + SEO optimization)
 - Progressive Web App (PWA)
 - Real-time updates (Socket.io / Firebase)
 - Razorpay payment integration
 - Google Maps API
-

2. Frontend Architecture

2.1 Recommended Tech Stack

Framework: Next.js 14+ (App Router)

Language: TypeScript

State Management: Zustand / Redux Toolkit

Styling: Tailwind CSS + shadcn/ui components

Forms: React Hook Form + Zod validation

API Layer: Axios / Fetch with React Query

Real-time: Socket.io-client

PWA: next-pwa

Notifications: Firebase Cloud Messaging

Maps: @react-google-maps/api

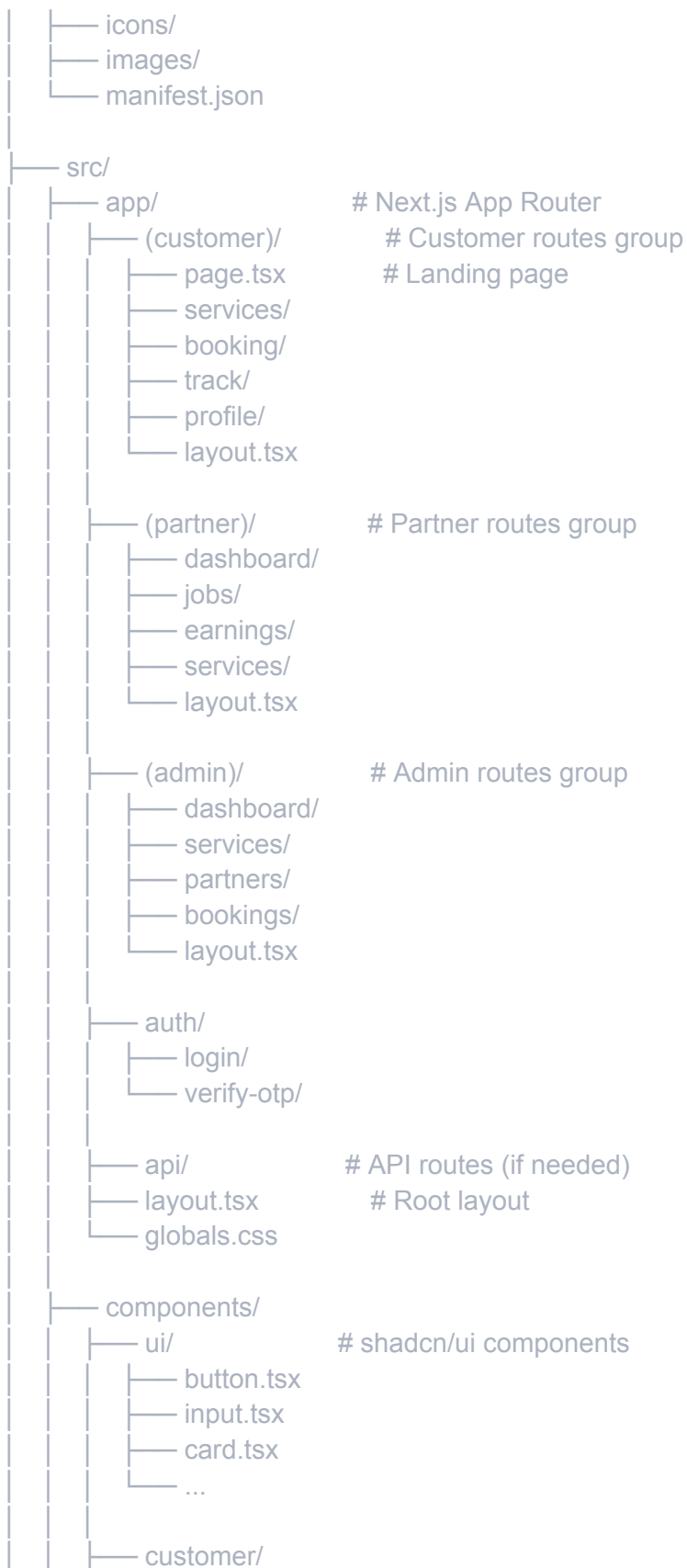
Payments: Razorpay Web SDK

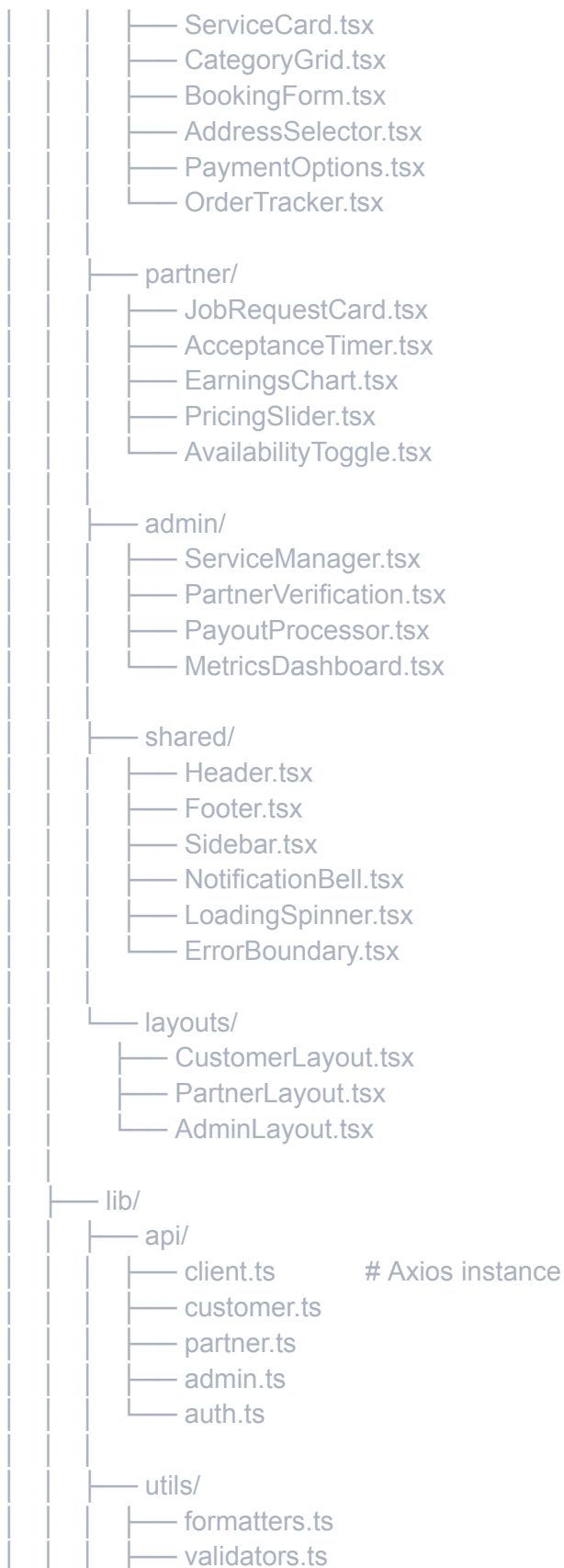
Testing: Jest + React Testing Library + Playwright

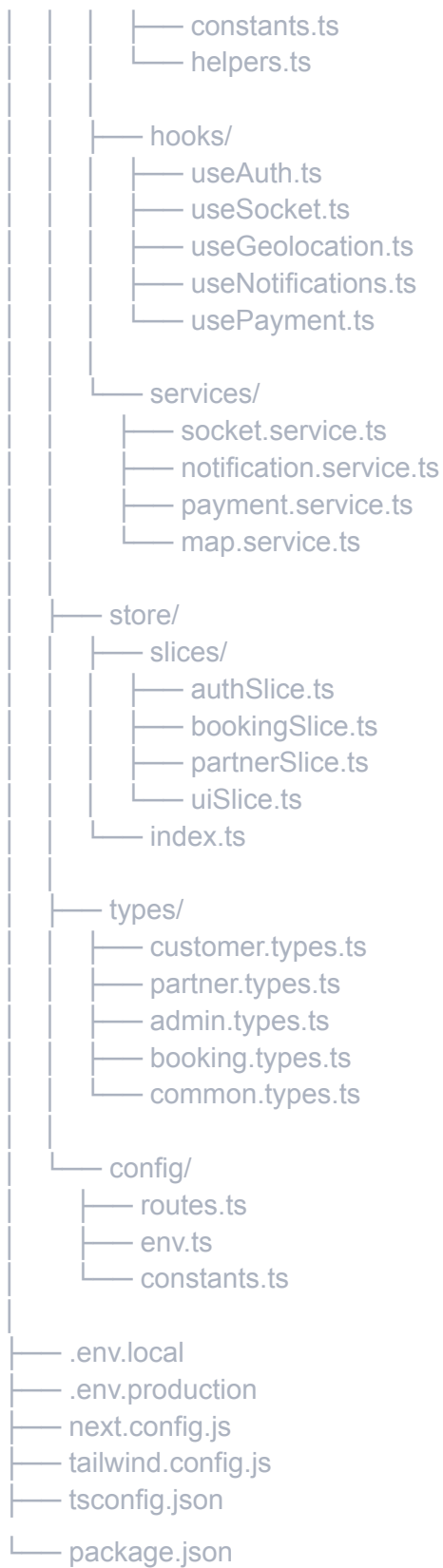
2.2 Project Structure

groomsta-frontend/

├── public/







3. Key Features Implementation Guide

3.1 Authentication System

Components:

- `LoginForm.tsx` - Phone/email input
- `OTPVerification.tsx` - OTP input with countdown
- `AuthProvider.tsx` - Context wrapper

Implementation Checklist:

- OTP-based phone authentication
- Optional email/password login
- Auto-create account on first OTP verification
- Guest browsing (no auth required)
- Auth persistence (JWT in httpOnly cookies)
- Role-based routing (customer/partner/admin)
- Auto-redirect based on user type

Key Hooks:

typescript

`useAuth()` → { user, login, logout, verifyOTP, isAuthenticated }

`useRoleRedirect()` → Auto-redirect based on role

3.2 Customer Flow Components

A. Service Discovery

Components:

- `ServiceCategoryGrid.tsx` - Visual category cards
- `ServiceSubcategoryList.tsx` - Subcategory navigation
- `ServiceCard.tsx` - Individual service display
- `ServiceDetailModal.tsx` - Full service information
- `SearchBar.tsx` - Service search with autocomplete

Features:

- Category → Subcategory → Service hierarchy
- Gender/location variant display

- Price range indication
- Duration estimates
- Customer reviews integration
- Add-ons and bundles
- "Add to Booking" action

B. Location & Address

Components:

- `PincodeChecker.tsx` - Serviceability verification
- `AddressForm.tsx` - Manual address input
- `MapPicker.tsx` - Google Maps integration
- `SavedAddresses.tsx` - Address management

Features:

- Pincode validation API call
- Google Maps autocomplete
- Pin location on map
- Save multiple addresses
- Set default address
- Edit/delete addresses

C. Booking Flow

Components:

- `DateTimePicker.tsx` - Calendar + time slots
- `BookingSummary.tsx` - Review before confirm
- `PaymentSelector.tsx` - Payment method options
- `BookingConfirmation.tsx` - Success screen

Features:

- Date selection with availability
- Time window selection (Morning/Afternoon/Evening)
- Dynamic pricing display (range → exact)
- Multiple payment options
- Razorpay integration
- Wallet balance application
- Coupon code entry
- Booking ID generation

D. Order Tracking

Components:

- `OrderTracker.tsx` - Status timeline
- `PartnerCard.tsx` - Partner information
- `ContactPartner.tsx` - Call/message options
- `RatingReview.tsx` - Post-service feedback

Features:

- Real-time status updates via WebSocket
- Visual status indicators (Accepted → On Way → Started → Completed)
- Partner details display
- Contact partner functionality
- Live map tracking (Phase 2)
- Post-service rating/review
- Payment completion (if pending)

E. Referral & Membership

Components:

- `ReferralCard.tsx` - Referral code sharing
- `MembershipPlans.tsx` - Plan comparison
- `WalletBalance.tsx` - Credits display

Features:

- Unique referral code generation
 - Share via WhatsApp/SMS/Copy link
 - Track pending/successful referrals
 - ₹100 auto-credit on successful referral
 - Membership plan display
 - Monthly subscription payment
 - Auto-applied membership discounts
-

3.3 Partner Flow Components

A. Partner Dashboard

Components:

- `JobRequestModal.tsx` - Incoming request with 20s timer
- `ActiveJobCard.tsx` - Ongoing job details

- `JobHistoryList.tsx` - Past jobs
- `EarningsSummary.tsx` - Weekly/monthly earnings
- `AvailabilityToggle.tsx` - Online/offline switch

Features:

- Real-time job request notifications
- 20-second countdown timer
- Accept/Reject actions
- Current week earnings
- Payout history and schedule
- Commission breakdown
- Toggle availability

B. Job Acceptance

Components:

- `JobRequestCard.tsx` - Request details
- `AcceptanceTimer.tsx` - Countdown display
- `JobDetailsView.tsx` - Full job information

Features:

- Push notification integration
- SMS alert backup
- Display: service, location, distance, payment
- Accept button (one-click)
- Reject button (no penalty)
- Auto-expire after 20 seconds
- Navigation link to customer location

C. Job Execution

Components:

- `JobStatusButtons.tsx` - Status update controls
- `CustomerContactCard.tsx` - Customer information
- `ServiceChecklist.tsx` - Services to perform
- `PaymentCollection.tsx` - Post-service payment

Features:

- "On the Way" button
- "Service Started" button (with timer)

- "Service Completed" button
- Customer contact details
- Map navigation link
- Upload completion photo (optional)
- Cash collection confirmation
- Earnings update in real-time

D. Pricing Management

Components:

- `ServicePricingList.tsx` - All services with pricing
- `PriceSlider.tsx` - Adjust within allowed range
- `AddOnManager.tsx` - Enable/disable add-ons

Features:

- View base price and allowed range
- Visual slider for price adjustment
- Compare with other partners
- Save pricing changes
- Manage custom add-ons
- Enable/disable services

E. Partner Onboarding

Components:

- `OnboardingWizard.tsx` - Multi-step form
- `KYCUUpload.tsx` - Document upload
- `BankDetailsForm.tsx` - Banking information
- `ServiceSelection.tsx` - Choose services to offer
- `PricingSetup.tsx` - Initial pricing

Features:

- Step-by-step wizard (5-6 steps)
- Personal details form
- OTP verification
- ID proof upload (Aadhaar/PAN/DL)
- Photo verification
- Experience and skills input
- Bank account details
- Salon details (if applicable)
- Service selection with pricing

- Submit for admin review
 - Application status tracking
-

3.4 Admin Dashboard Components

A. Overview

Components:

- `MetricsDashboard.tsx` - Key KPIs
- `RevenueChart.tsx` - Revenue visualization
- `ActivityFeed.tsx` - Recent activities

Features:

- Today's bookings count
- Active partners count
- Revenue summary
- Real-time metrics updates
- Graphical representations

B. Service Management

Components:

- `ServiceCRUD.tsx` - Create/edit services
- `CategoryManager.tsx` - Manage categories
- `PricingConfigurator.tsx` - Set base prices and ranges

Features:

- Add/edit/delete services
- Category and subcategory management
- Set base prices
- Define partner price ranges ($\pm\%$)
- Add service variants
- Enable/disable services
- Reorder categories

C. Partner Management

Components:

- `PartnerVerificationQueue.tsx` - Pending applications
- `PartnerList.tsx` - All partners
- `PartnerDetails.tsx` - Individual partner view
- `PerformanceMetrics.tsx` - Partner analytics

Features:

- View pending verifications
- Review KYC documents
- Approve/reject applications
- Request additional information
- Suspend/activate partners
- View partner earnings
- Performance metrics (acceptance rate, ratings)

D. Booking Management

Components:

- `BookingList.tsx` - All bookings with filters
- `BookingDetails.tsx` - Individual booking view
- `DisputeResolver.tsx` - Handle disputes

Features:

- View all bookings (active/completed/cancelled)
- Filter by status, date, partner
- View booking details
- Handle customer complaints
- Process refunds
- Manage disputed bookings

E. Payout Processing

Components:

- `PayoutProcessor.tsx` - Weekly payout interface
- `PayoutReport.tsx` - Detailed payout report
- `CommissionTracker.tsx` - Commission analytics

Features:

- Generate weekly payout report
- Calculate partner earnings
- Deduct commissions

- Verify bank details
 - Initiate batch transfers
 - Monitor payout status
 - Archive payout records
 - Send notifications to partners
-

4. State Management Strategy

4.1 Global State (Zustand/Redux)

typescript

// Auth Store

```
{
  user: User | null,
  role: 'customer' | 'partner' | 'admin',
  isAuthenticated: boolean,
  token: string | null
}
```

// Booking Store (Customer)

```
{
  currentBooking: {
    services: Service[],
    address: Address | null,
    dateTime: DateTime | null,
    payment: PaymentOption | null
  },
  activeBookings: Booking[],
  bookingHistory: Booking[]
}
```

// Partner Store

```
{
  incomingRequests: JobRequest[],
  activeJobs: Job[],
  earnings: {
    currentWeek: number,
    payoutHistory: Payout[]
  },
  availability: boolean,
  services: PartnerService[]
}
```

```
// UI Store
{
  notifications: Notification[],
  loading: boolean,
  modals: { [key: string]: boolean },
  toast: Toast | null
}
```

4.2 Server State (React Query)

```
typescript
// Use React Query for API calls
useQuery(['services', categoryId], fetchServices)
useQuery(['booking', bookingId], fetchBooking)
useMutation(createBooking, { onSuccess: ... })
```

5. Real-time Features

5.1 WebSocket Integration

Events to Handle:

Customer Side:

- `booking_accepted` - Partner assigned
- `partner_on_way` - Partner traveling
- `service_started` - Service began
- `service_completed` - Service finished
- `partner_cancelled` - Need reassignment

Partner Side:

- `new_job_request` - Incoming booking (20s window)
- `job_assigned` - Job confirmed to partner
- `job_cancelled` - Customer cancelled
- `customer_message` - Customer sent message

Implementation:

```
typescript
```

```
// useSocket.ts
const socket = io(SOCKET_URL);

socket.on('new_job_request', (data) => {
  // Show JobRequestModal
  // Start 20-second timer
});

socket.on('booking_accepted', (data) => {
  // Update booking status
  // Show partner details
});
```

6. Payment Integration

6.1 Razorpay Implementation

Components:

- `RazorpayCheckout.tsx` - Payment gateway wrapper
- `PaymentStatus.tsx` - Success/failure handling

Flow:

1. User selects payment option
2. If online payment → Open Razorpay checkout
3. Handle success/failure callbacks
4. Update booking status
5. Show confirmation

Key Features:

- Full online payment
 - Partial payment (advance)
 - Post-service payment
 - Wallet integration
 - Refund handling
 - Payment receipts
-

7. PWA Implementation

7.1 PWA Features

Configuration:

```
javascript
// next.config.js
const withPWA = require('next-pwa')({
  dest: 'public',
  register: true,
  skipWaiting: true,
  disable: process.env.NODE_ENV === 'development'
});
...

```

****Features Checklist:****

- [] Manifest.json configuration
- [] Service worker setup
- [] Add to home screen prompt
- [] Offline page caching
- [] Background sync for actions
- [] Push notification setup
- [] App-like navigation
- [] Splash screen

8. Responsive Design Guidelines

8.1 Breakpoints (Tailwind)

...

sm: 640px // *Mobile landscape*

md: 768px // *Tablet*

lg: 1024px // *Laptop*

xl: 1280px // *Desktop*

2xl: 1536px // *Large desktop*

8.2 Mobile-First Approach

- Design for mobile (375px) first
- Progressive enhancement for larger screens
- Touch-friendly UI elements (min 44px tap targets)

- Bottom navigation for mobile
- Simplified forms on mobile

8.3 Key Responsive Patterns

- Collapsible sidebar on mobile
 - Bottom sheet modals on mobile
 - Grid to stack layout transitions
 - Hamburger menu for mobile nav
 - Swipe gestures for cards
-

9. Performance Optimization

9.1 Code Splitting

typescript

// Dynamic imports for heavy components

```
const MapPicker = dynamic(() => import('@/components/customer/MapPicker'), {  
  loading: () => <LoadingSpinner />,  
  ssr: false  
});  
...
```

9.2 Image Optimization

- Use Next.js `<Image>` component
- Implement lazy loading
- Serve WebP format
- Proper sizing and srcset

9.3 Bundle Optimization

- [] Tree shaking unused code
- [] Code splitting by route
- [] Lazy load below-fold components
- [] Minimize third-party scripts
- [] Use React.memo for expensive components

9.4 Performance Metrics Target

- First Contentful Paint (FCP): < 1.8s
- Largest Contentful Paint (LCP): < 2.5s
- Time to Interactive (TTI): < 3.8s
- Cumulative Layout Shift (CLS): < 0.1

10. Security Best Practices

10.1 Authentication

- [] JWT stored in httpOnly cookies
- [] CSRF token implementation
- [] XSS prevention (sanitize inputs)
- [] Rate limiting on auth endpoints
- [] Secure OTP generation

10.2 Data Protection

- [] Input validation (client + server)
- [] Sanitize user-generated content
- [] Secure payment data handling
- [] HTTPS enforcement
- [] Environment variables for secrets

10.3 API Security

- [] Authorization headers
- [] Request signing
- [] API rate limiting
- [] CORS configuration
- [] Error message sanitization

11. Accessibility (A11y)

11.1 WCAG 2.1 Compliance

- [] Semantic HTML elements
- [] ARIA labels where needed
- [] Keyboard navigation support
- [] Focus indicators visible
- [] Screen reader compatible
- [] Color contrast ratio $\geq 4.5:1$
- [] Alt text for images
- [] Form labels and error messages

11.2 Key Areas

- [] Skip to main content link
- [] Focus management in modals
- [] Loading states announced
- [] Error states clear and actionable

- [] Touch targets $\geq 44\text{px}$

12. Testing Strategy

12.1 Unit Tests (Jest)

- [] Utility functions
- [] Custom hooks
- [] Form validation logic
- [] State management slices

12.2 Component Tests (RTL)

- [] Component rendering
- [] User interactions
- [] Props handling
- [] Error boundaries

12.3 Integration Tests

- [] Complete user flows
- [] API integration
- [] Payment flows
- [] Real-time features

12.4 E2E Tests (Playwright)

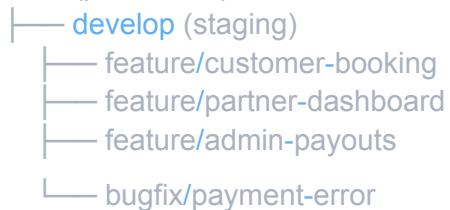
- [] Customer booking flow
- [] Partner job acceptance
- [] Payment processing
- [] Authentication flows

13. Development Workflow

13.1 Git Strategy

...

main (production)



13.2 Code Standards

- ESLint configuration
- Prettier formatting
- Husky pre-commit hooks
- TypeScript strict mode
- Conventional commits

13.3 CI/CD Pipeline

- Automated testing on PR
 - Build verification
 - Lighthouse CI for performance
 - Automated deployment to staging
 - Manual approval for production
-

14. Key Technical Decisions

14.1 Why Next.js?

✓ SEO optimization (SSR/SSG) ✓ API routes (optional backend) ✓ Image optimization built-in ✓ File-based routing ✓ Great PWA support

14.2 Why TypeScript?

✓ Type safety ✓ Better IDE support ✓ Catch errors early ✓ Self-documenting code ✓ Easier refactoring

14.3 Why Tailwind CSS?

✓ Rapid development ✓ Consistent design system ✓ Small bundle size ✓ Mobile-first approach ✓ Easy customization

15. Development Checklist

Phase 1: Foundation (Week 1-2)

- Project setup with Next.js + TypeScript
- Install and configure dependencies

- Setup Tailwind CSS + shadcn/ui
- Configure ESLint + Prettier
- Setup folder structure
- Create base layouts
- Configure environment variables
- Setup API client (Axios)
- Implement authentication flow
- Setup state management

Phase 2: Customer Flow (Week 3-4)

- Landing page
- Service discovery (categories/subcategories)
- Service detail pages
- Location and address components
- Booking flow components
- Payment integration
- Order tracking
- Referral system
- Membership features

Phase 3: Partner Flow (Week 5-6)

- Partner onboarding wizard
- Partner dashboard
- Job request notifications
- Job acceptance with timer
- Active job management
- Status update controls
- Pricing management
- Earnings tracking
- Payout history

Phase 4: Admin Dashboard (Week 7)

- Admin authentication
- Metrics dashboard
- Service management CRUD
- Partner verification queue
- Booking management
- Payout processing
- Reports and analytics

Phase 5: Real-time & Notifications (Week 8)

- WebSocket integration
- Push notifications setup
- SMS notifications
- WhatsApp notifications
- Email templates
- Notification preferences

Phase 6: PWA & Polish (Week 9-10)

- PWA configuration
- Offline capabilities
- Add to home screen
- Performance optimization
- Accessibility audit
- Cross-browser testing
- Mobile responsiveness
- Error handling
- Loading states

Phase 7: Testing & Launch (Week 11-12)

- Unit tests
- Integration tests
- E2E tests
- Security audit
- Performance testing
- User acceptance testing
- Bug fixes
- Documentation
- Production deployment

16. Key Dependencies

json

```
{  
  "dependencies": {  
    "next": "^14.0.0",  
    "react": "^18.2.0",  
    "react-dom": "^18.2.0",  
    "typescript": "^5.0.0",  
    "@tanstack/react-query": "^5.0.0",  
    "zustand": "^4.4.0",
```

```
"socket.io-client": "^4.6.0",
"axios": "^1.6.0",
"react-hook-form": "^7.48.0",
"zod": "^3.22.0",
"@react-google-maps/api": "^2.19.0",
"date-fns": "^2.30.0",
"recharts": "^2.10.0",
"lucide-react": "^0.300.0",
"next-pwa": "^5.6.0",
"firebase": "^10.7.0"
},
"devDependencies": {
  "@types/node": "^20.0.0",
  "@types/react": "^18.2.0",
  "eslint": "^8.55.0",
  "prettier": "^3.1.0",
  "tailwindcss": "^3.4.0",
  "jest": "^29.7.0",
  "@testing-library/react": "^14.1.0",
  "@playwright/test": "^1.40.0",
  "husky": "^8.0.0"
}
}
```

17. Environment Variables

env

```
# .env.local
NEXT_PUBLIC_API_URL=
NEXT_PUBLIC_SOCKET_URL=
NEXT_PUBLIC_RAZORPAY_KEY=
NEXT_PUBLIC_GOOGLE_MAPS_KEY=
NEXT_PUBLIC_FIREBASE_API_KEY=
NEXT_PUBLIC_FIREBASE_AUTH_DOMAIN=
NEXT_PUBLIC_FIREBASE_PROJECT_ID=

# Server-side only
RAZORPAY_SECRET=
JWT_SECRET=
DATABASE_URL=
AWS_S3_BUCKET=
```

18. Critical UI/UX Considerations

18.1 Customer Experience

- **Speed is critical:** Booking should complete in < 2 minutes
- **Trust building:** Show partner details, ratings, photos
- **Price transparency:** Always show price range upfront
- **Status clarity:** Clear visual indicators for booking status
- **Easy cancellation:** Simple refund process

18.2 Partner Experience

- **Quick acceptance:** One-tap to accept job
- **Clear earnings:** Always show exact earnings upfront
- **Minimal friction:** Reduce steps to complete job
- **Reliable notifications:** Never miss a job request
- **Fair pricing:** Flexibility within limits

18.3 Admin Experience

- **Efficiency:** Bulk actions where possible
- **Quick verification:** Streamlined approval process
- **Visual analytics:** Charts and graphs for metrics
- **Quick access:** Most-used features prominent
- **Error prevention:** Confirmations for critical actions

19. Common Pitfalls to Avoid

✗ Don't:

- Hardcode API URLs (use env variables)
- Skip loading states (causes poor UX)
- Ignore error boundaries (app crashes)
- Forget mobile testing (most users)
- Skip accessibility (legal requirement)
- Ignore security (XSS, CSRF)
- Over-optimize prematurely (YAGNI principle)
- Skip TypeScript types (defeats the purpose)

 Do:

- Use environment variables
 - Show loading indicators
 - Implement error boundaries
 - Test on real devices
 - Follow WCAG guidelines
 - Sanitize all inputs
 - Optimize when needed
 - Properly type everything
-

20. Monitoring & Analytics

20.1 Performance Monitoring

- Setup Google Analytics / Mixpanel
- Real User Monitoring (RUM)
- Error tracking (Sentry)
- Performance metrics tracking

20.2 Key Metrics to Track

Customer:

- Conversion rate (visit → booking)
- Booking completion time
- Payment success rate
- Cancellation rate

Partner:

- Acceptance rate
- Average response time
- Jobs completed per week
- Earnings growth

Platform:

- Active users (daily/weekly)
- GMV (Gross Merchandise Value)
- Commission revenue
- System uptime

21. Final Pre-Launch Checklist

21.1 Functionality

- All user flows work end-to-end
- Payment integration tested with real transactions
- Real-time notifications working
- OTP sending working
- All forms validate correctly
- Error messages are user-friendly

21.2 Performance

- Lighthouse score > 90
- Page load time < 3s
- Images optimized
- Bundle size analyzed
- Mobile performance tested

21.3 Security

- All API endpoints authenticated
- Input validation on all forms
- XSS protection implemented
- HTTPS enforced
- Rate limiting configured

21.4 Design

- Responsive on all devices
- Cross-browser tested
- Consistent branding
- No broken images/links
- Loading states everywhere

21.5 Legal & Compliance

- Terms of Service page
- Privacy Policy page
- Cookie consent banner
- GDPR compliance (if applicable)
- Refund policy clear

22. Post-Launch Monitoring

Week 1:

- Monitor error rates hourly
- Check payment success rates
- Track partner acceptance rates
- Watch for performance degra

Architected comprehensive frontend documentation spanning architecture, workflows, and deployment strategies.

The user wants me to create a comprehensive frontend structure document, guidelines, and checklist based on the Groomsta app flow and PRD documents they provided. I've created a detailed document covering:

1. Project overview and tech stack
2. Detailed folder structure
3. Component breakdown for all three flows (Customer, Partner, Admin)
4. State management strategy
5. Real-time features implementation
6. Payment integration
7. PWA implementation
8. Responsive design
9. Performance optimization
10. Security best practices
11. Accessibility
12. Testing strategy
13. Development workflow
14. Technical decisions
15. Phase-wise development checklist
16. Dependencies
17. Environment variables
18. UI/UX considerations
19. Common pitfalls
20. Monitoring and analytics
21. Pre-launch checklist
22. Post-launch monitoring

The document is comprehensive and follows senior-level frontend architecture best practices. I should continue with the post-launch section and wrap up.

dation

Month 1:

- Analyze user feedback
- A/B test key flows
- Optimize conversion funnels
- Fix critical bugs

Month 3:

- Review analytics data
 - Plan feature improvements
 - Evaluate partner satisfaction
 - Customer satisfaction survey
-

23. Contact & Documentation

23.1 Code Documentation

- Component prop documentation (JSDoc)
- API endpoint documentation
- State management documentation
- Custom hooks documentation
- Setup instructions (README.md)

23.2 Design System Documentation

- Color palette
- Typography scale
- Component library
- Spacing system
- Icon library

23.3 Deployment Documentation

- Environment setup guide
 - Build and deployment steps
 - Environment variables reference
 - Rollback procedures
 - Monitoring setup
-

Summary

This frontend structure provides a scalable, maintainable foundation for Groomsta's web application. Key highlights:

✓ **Modular Architecture:** Clear separation of customer, partner, and admin flows ✓ **Type Safety:** Full TypeScript implementation ✓ **Performance:** PWA capabilities, code splitting, optimization ✓ **Real-time:** WebSocket integration for instant updates ✓ **Scalability:** Built to handle growth ✓ **User Experience:** Mobile-first, accessible, intuitive ✓ **Security:** Best practices implemented throughout ✓ **Testing:** Comprehensive testing strategy ✓ **Maintainability:** Clear structure, documented code

Estimated Timeline: 12 weeks for MVP **Team Size:** 2-3 frontend developers **Tech Stack:** Next.js, TypeScript, Tailwind CSS, Socket.io

Document Version: 1.0

Last Updated: December 2024

Status: Ready for Development

Next Steps: Begin Phase 1 implementation